Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине «АПО ЭВМиС»
Тема: «Изучение аппаратно-программной архитектуры процессоров семейства Intel – разработка ассемблерной вставки».

Выполнил:
Студент 4-го курса
Группы АС-66
Ляшук В.И.
Проверила:
Кулик А.Д.

Брест 2026

Цель работы: изучить программную модель микропроцессора, регистры и организацию памяти; получить практические навыки работы с ассемблерными вставками.

Вариант №12

Задание: заменить пробелы на табы. Обработку строки оформить в виде отдельной функции с ассемблерной вставкой, реализовать ту же обработку функцией на C++ и сравнить быстродействие обеих реализаций. Среда разработки – MS Visual Studio (платформа x86).

Ход работы

```
#include <Windows.h>
#include <iostream>
#include <string>
#include <chrono>
#include <iomanip>
#include <algorithm>

// Замена пробелов на C++
__declspec(noinline)
void replaceCpp(char* text, unsigned int length)
{
    for (unsigned int i = 0; i < length; ++i)
    {
        if (text[i] == ' ')
            text[i] = '\t';
    }
}

__declspec(noinline)
void replaceAsm(char* buffer, unsigned int countChars)
{
    __asm
    {
        mov edx, buffer
        mov ecx, countChars

        test ecx, ecx
        jz asm_done

        asm_loop :
        cmp byte ptr[edx], 20h
            jne asm_next

            mov byte ptr[edx], 09h

            asm_next :
        inc edx
            dec ecx
            jnz asm_loop

            asm_done :
        nop
    }
}

// Отображение табуляций в виде \t для проверки
void printVisible(const std::string& text)
{
    for (char c : text)
    {
        if (c == '\t')
            std::cout << "\\t";
        else
            std::cout << c;
    }
    std::cout << '\n';
```

```
}

int main()
{
    SetConsoleCP(CP_UTF8);
    SetConsoleOutputCP(CP_UTF8);

    std::cout << "Лабораторная работа. Задание №12\n";
    std::cout << "Заменить пробелы на символ табуляции.\n\n";

    std::string original;

    std::cout << "Введите строку: ";
    std::getline(std::cin, original);

    std::string cppResult = original;
    std::string asmResult = original;

    replaceCpp(cppResult.data(),
        static_cast<unsigned int>(cppResult.size()));

    replaceAsm(asmResult.data(),
        static_cast<unsigned int>(asmResult.size()));

    std::cout << "\nИсходная строка: [" << original << "]\n";
    std::cout << "Результат C++:  [" << cppResult << "]\n";
    std::cout << "Результат ASM:  [" << asmResult << "]\n";

    std::cout << "\nТабуляции показаны как \\t:\n";
    std::cout << "C++: ";
    printVisible(cppResult);

    std::cout << "ASM: ";
    printVisible(asmResult);

    std::cout << "\nРезультаты совпадают: "
        << (cppResult == asmResult ? "да" : "нет") << '\n';

    // Большой текст для измерения быстродействия
    const unsigned int SIZE = 8 * 1024 * 1024;
    const int REPEATS = 30;

    std::string testText(SIZE, 'a');

    for (unsigned int i = 0; i < SIZE; i += 5)
        testText[i] = ' ';

    std::string cppBuffer = testText;
    std::string asmBuffer = testText;

    using Clock = std::chrono::steady_clock;

    double cppTime = 0.0;
    double asmTime = 0.0;

    // Прогрев
    replaceCpp(cppBuffer.data(), SIZE);
    replaceAsm(asmBuffer.data(), SIZE);

    auto measureCpp = [&]()
        {
            // Подготовка данных не входит в измерение
            std::copy(testText.begin(), testText.end(), cppBuffer.begin());

            const auto start = Clock::now();
            replaceCpp(cppBuffer.data(), SIZE);
            const auto end = Clock::now();

            cppTime +=
                std::chrono::duration<double, std::milli>(end - start).count();
        };

    auto measureAsm = [&]()
        {
            std::copy(testText.begin(), testText.end(), asmBuffer.begin());

            const auto start = Clock::now();
            replaceAsm(asmBuffer.data(), SIZE);
```

```
            const auto end = Clock::now();

            asmTime +=
                std::chrono::duration<double, std::milli>(end - start).count();
        };

    for (int i = 0; i < REPEATS; ++i)
    {
        // Чередование порядка измерений
        if (i % 2 == 0)
        {
            measureCpp();
            measureAsm();
        }
        else
        {
            measureAsm();
            measureCpp();
        }

        if (cppBuffer != asmBuffer)
        {
            std::cout << "Ошибка: результаты обработки отличаются.\n";
            return 1;
        }
    }

    cppTime /= REPEATS;
    asmTime /= REPEATS;

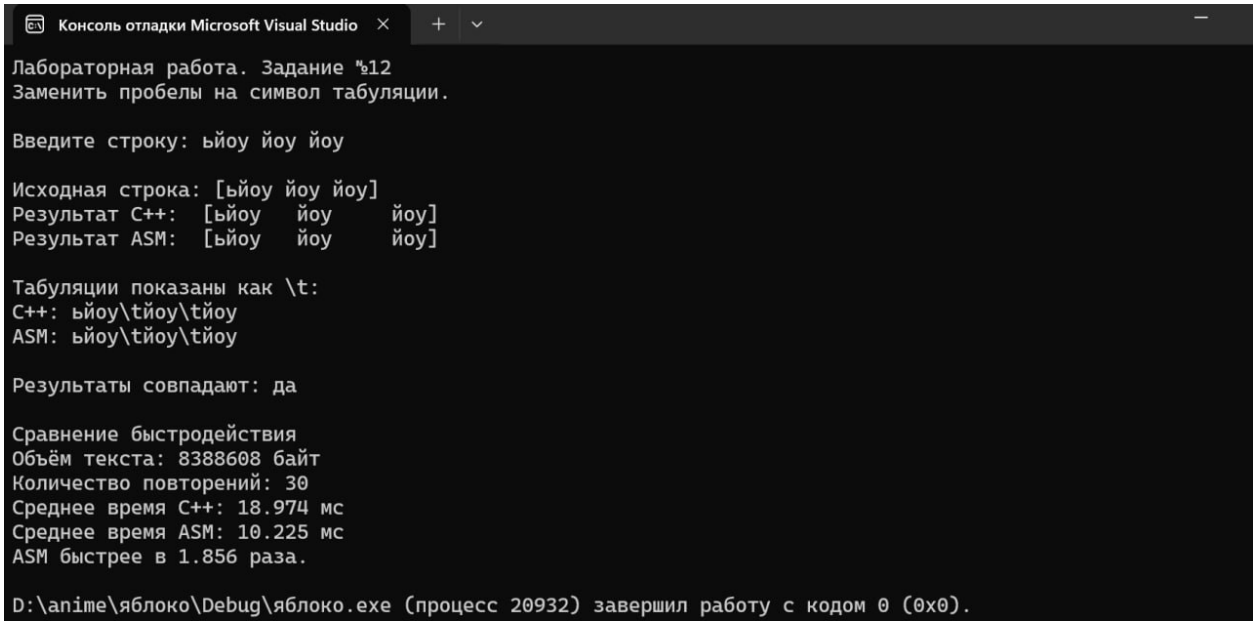
    std::cout << std::fixed << std::setprecision(3);

    std::cout << "\nСравнение быстродействия\n";
    std::cout << "Объём текста: " << SIZE << " байт\n";
    std::cout << "Количество повторений: " << REPEATS << '\n';
    std::cout << "Среднее время C++: " << cppTime << " мс\n";
    std::cout << "Среднее время ASM: " << asmTime << " мс\n";

    if (cppTime > 0.0 && asmTime > 0.0)
    {
        if (cppTime < asmTime)
        {
            std::cout << "C++ быстрее в "
                << asmTime / cppTime << " раза.\n";
        }
        else if (asmTime < cppTime)
        {
            std::cout << "ASM быстрее в "
                << cppTime / asmTime << " раза.\n";
        }
        else
        {
            std::cout << "Время выполнения одинаковое.\n";
        }
    }

    return 0;
}
```

Результат:



Результат работы программы

Вывод: Я изучил программную модель микропроцессора, регистры и организацию памяти; получил практические навыки работы с ассемблерными вставками.